SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

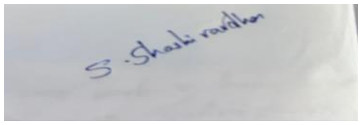
Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.
Design a JDBC- based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I S.Shashi vardhan reddy 192311155(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1: Servlet-Based Student Registration System
Architecture of a Servlet-Based Application

Client (Browser)
  │ HTTP Request (GET/POST)
  ▼
Web Server / Servlet Container (e.g., Apache Tomcat)
  │ 1. Reads web.xml / annotations to map URL → Servlet
  │ 2. Creates request/response objects
  ▼
Servlet (Business Logic Layer)
  │ Processes form data, validates input
  ▼
Model / DAO Layer (optional, talks to DB via JDBC)
  ▼
Response generated (HTML / JSON) sent back to Client

Key components:

- **Client**: Browser submits the registration form.
- **Servlet Container**: Manages servlet lifecycle, threading, request dispatching (Tomcat, Jetty).

- **Servlet**: A Java class extending `HttpServlet` that intercepts requests via `doGet()`/`doPost()`.
- **web.xml / @WebServlet annotation**: Maps a URL pattern to a servlet class.
- **Deployment Descriptor**: Configures servlet mappings, session timeout, filters.

This follows the **Model-View-Controller** pattern loosely — Servlet acts as Controller, JSP/HTML as View, and JavaBeans/DAO as Model.

Servlet Implementation — Student Registration Form

registration.html

```
<!DOCTYPE html>
<html>
<body>
<h2>Student Registration</h2>
<form action="register" method="post">
   Name: <input type="text" name="name" required><br>
   Email: <input type="email" name="email" required><br>
   Course: <input type="text" name="course" required><br>
   <input type="submit" value="Register">
</form>
</body>
</html>
```

RegistrationServlet.java

```
import java.io.*;
```

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/register")
public class RegistrationServlet extends HttpServlet {

   // Called once when servlet is loaded
   @Override
   public void init() throws ServletException {
      System.out.println("RegistrationServlet initialized");
      // Ideal place to initialize DB connection pool, config, etc.
   }

   // Handles GET requests (e.g., pre-filling a form, search by roll no.)
   @Override
   protected void doGet(HttpServletRequest request, HttpServletResponse
response)
         throws ServletException, IOException {
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("<h3>Please use the registration form to submit your details
(POST).</h3>");
   }

   // Handles POST requests (actual form submission)
   @Override
   protected void doPost(HttpServletRequest request, HttpServletResponse
response)
         throws ServletException, IOException {

      // Read form parameters
      String name = request.getParameter("name");
      String email = request.getParameter("email");
      String course = request.getParameter("course");

      // Basic validation
      if (name == null || name.trim().isEmpty()) {
         response.sendError(HttpServletResponse.SC_BAD_REQUEST,      "Name    is
required");
         return;
      }

      // In real app: call DAO layer to insert into DB (see Q3)
```

```
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h3>Registration Successful</h3>");
        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Email: " + email + "</p>");
        out.println("<p>Course: " + course + "</p>");
    }

    // Called once when servlet is being removed from service
    @Override
    public void destroy() {
        System.out.println("RegistrationServlet destroyed - cleanup resources here");
    }
}
```

Servlet Life Cycle

Phase	Method	When it Runs	Frequency
Loading & Instantiation	Constructor	Container loads the class and creates one instance	Once
Initialization	`init()`	Immediately after instantiation, before any request is served	Once
Request Handling	`service()` → `doGet()` / `doPost()`	On every incoming client request	Every request (multi-threaded)
Destruction	`destroy()`	When container shuts down or unloads the servlet	Once

The container creates **a single servlet instance** and spawns a **new thread per request**, calling `service()` on that shared instance — so instance variables must be handled carefully (avoid storing per-request state as instance fields).

- `service()` internally dispatches to `doGet()`, `doPost()`, `doPut()`, `doDelete()`, etc., based on the HTTP method.

GET vs POST for Sensitive Data

POST is more appropriate for sensitive data (e.g., passwords, personal identification, payment info) because:

1. **Data visibility**: GET appends parameters to the URL (`?name=John&email=...`), which gets logged in browser history, proxy logs, and server access logs. POST sends data in the request body, which is not exposed in the URL.
2. **URL length limits**: GET has practical length restrictions (~2048 characters depending on browser), unsuitable for larger form payloads.
3. **Caching**: GET requests can be cached by browsers/proxies, potentially exposing sensitive data; POST requests are not cached by default.
4. **Bookmarking/sharing risk**: URLs with GET parameters can be accidentally bookmarked or shared, leaking data.
5. **Idempotency semantics**: GET is meant for retrieval (safe, idempotent, no side effects), while POST is meant for actions that change server state (like creating a registration record) — using POST also aligns with correct REST semantics.

Note: POST alone doesn't guarantee security (data is still plaintext unless HTTPS is used) — **POST + HTTPS + server-side validation/sanitization** is the correct combination for handling sensitive registration data.

Question 2: Session Management with HttpSession and Cookies

How Session Tracking Works

HTTP is a **stateless protocol** — each request is independent and the server has no memory of previous requests by default. Session tracking is the mechanism used to associate a series of requests from the same client into a logical "session," typically achieved through:

1. **Cookies** – server sends a small piece of data to the browser, which the browser returns on every subsequent request.
2. **URL Rewriting** – session ID appended to every URL (used when cookies are disabled).
3. **Hidden Form Fields** – session ID embedded in hidden form inputs.
4. **HttpSession API** – built on top of cookies (or URL rewriting), servlet containers provide a high-level abstraction (`JSESSIONID`).

Flow with HttpSession

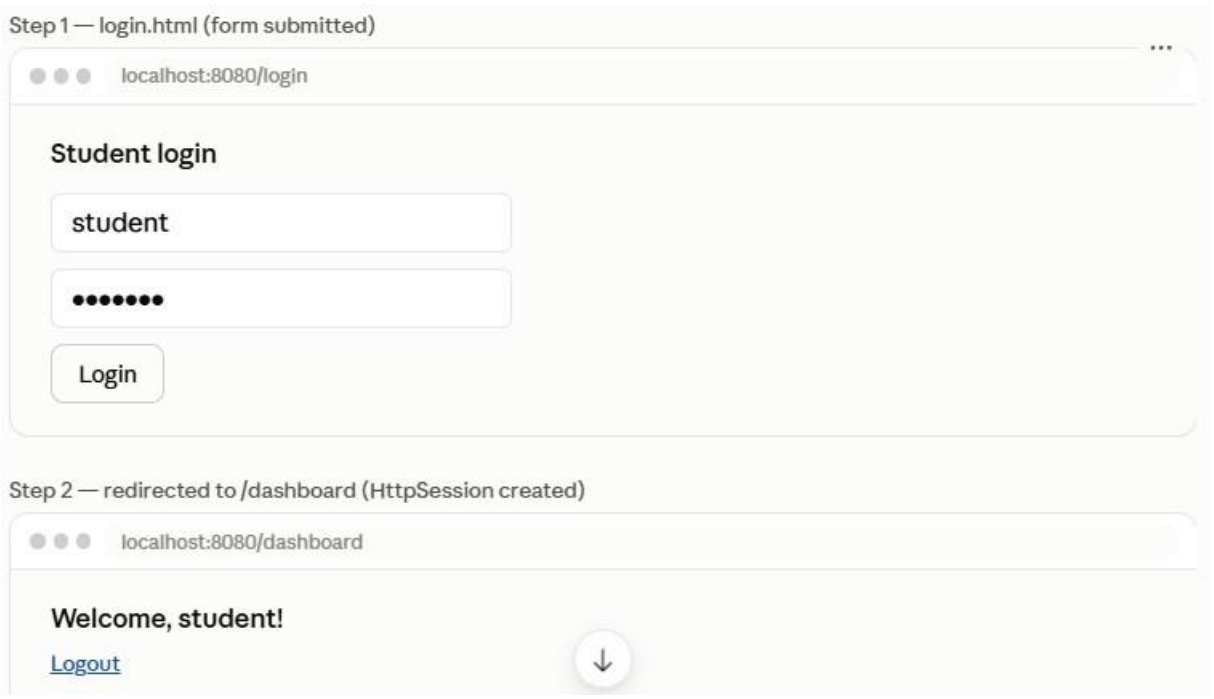
1. User submits login credentials (POST /login)
2. Servlet validates credentials
3. Servlet calls request.getSession() → container creates a new HttpSession and generates a unique session ID
4. Container sends this ID to browser as a cookie: JSESSIONID=ABC123
5. Browser stores the cookie and sends it automatically on every request to the same domain
6. Servlet retrieves session via request.getSession(false) on each page and reads/writes user-specific attributes

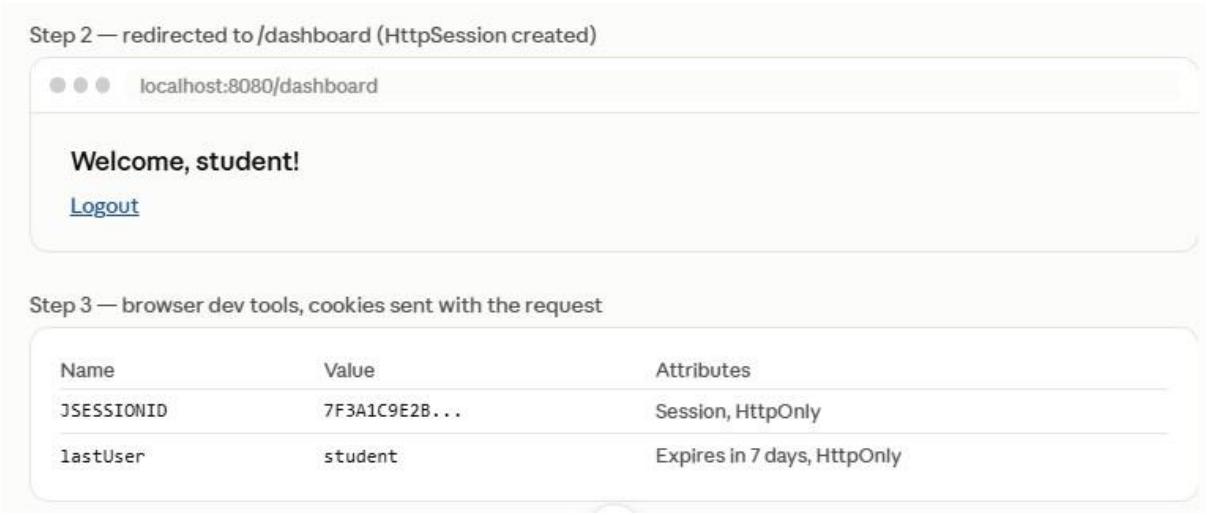
Implementation

LoginServlet.java

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

   protected void doPost(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {

      String username = request.getParameter("username");
      String password = request.getParameter("password");

      // Simplified authentication check (use DAO + hashed passwords in production)
      if (isValidUser(username, password)) {

         // Create session (true = create if not exists)
         HttpSession session = request.getSession(true);
         session.setAttribute("username", username);
         session.setAttribute("loginTime", new java.util.Date());
         session.setMaxInactiveInterval(30 * 60); // 30 minutes timeout

         // Optionally set a persistent cookie for "remember me"
         Cookie rememberCookie = new Cookie("lastUser", username);
         rememberCookie.setMaxAge(7 * 24 * 60 * 60); // 7 days
         rememberCookie.setHttpOnly(true); // not accessible via JavaScript
         response.addCookie(rememberCookie);

         response.sendRedirect("dashboard");
      } else {
         response.sendRedirect("login.html?error=true");
      }
   }
```

```
    private boolean isValidUser(String u, String p) {
        // Placeholder — replace with DB check via JDBC
        return "student".equals(u) && "pass123".equals(p);
    }
}
```





DashboardServlet.java (protected page checking session)

```
@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {
```

```
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false); // don't create new session

        if (session == null || session.getAttribute("username") == null) {
            response.sendRedirect("login.html"); // not authenticated
            return;
        }

        String username = (String) session.getAttribute("username");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h3>Welcome, " + username + "!</h3>");
        out.println("<a href='logout'>Logout</a>");
    }
}
```

LogoutServlet.java

```
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate(); // destroys session and all attributes
        }
        response.sendRedirect("login.html");
    }
}
```

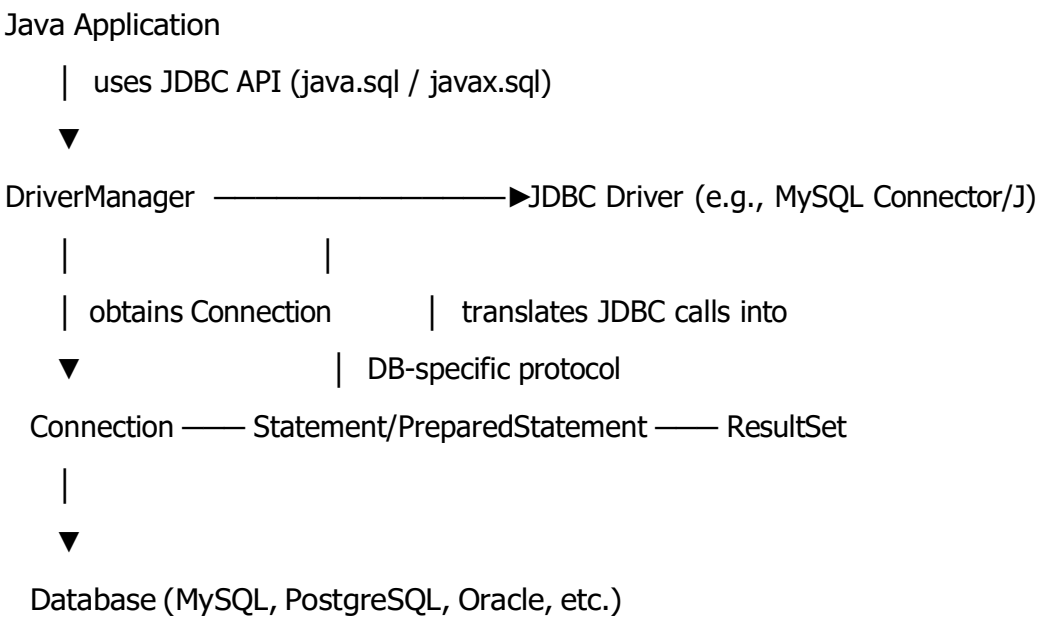
Aspect	HttpSession	Cookies
Storage location	Server-side (session data lives in server memory/store)	Client-side (stored in browser)
Data sent over network	Only session ID (`JSESSIONID`) is transmitted	Full cookie data is transmitted with every request
Security	More secure — sensitive data never leaves the server	Less secure — data visible/editable on client unless encrypted
Capacity	Can hold large/complex objects (limited by server memory)	Limited to ~4KB per cookie
Lifetime control	`setMaxInactiveInterval()`, invalidated on `session.invalidate()` or timeout	`setMaxAge()` — persists even after browser closes if positive value set
Dependency	Internally relies on a cookie (`JSESSIONID`) or URL rewriting to identify the session	Self-contained, no server-side storage needed
Use case	Storing login state, shopping cart, user preferences during active use	"Remember me" tokens, tracking preferences across visits, lightweight non-sensitive data
Scalability concern	Session replication needed in clustered/load-balanced environments	No such issue since data resides on client

Best practice: Use `HttpSession` for authentication state and sensitive data; use `Cookies` only for non-sensitive, longer-lived preferences (and always mark sensitive cookies `HttpOnly` + `Secure`).

3: JDBC-Based Product Management for E-Commerce

JDBC Architecture

JDBC (Java Database Connectivity) provides a standard API for Java applications to interact with relational databases, using a driver-based architecture:



Core components:

- **DriverManager**: Manages a list of database drivers and establishes connections.
  - **Driver**: Vendor-specific implementation that communicates with the actual database (Type 4 drivers are pure-Java, most common today).
  - **Connection**: Represents a session with the database.
  - **Statement / PreparedStatement / CallableStatement**: Used to execute SQL queries.
  - **ResultSet**: Holds tabular data returned by a query.
  - **SQLException**: Handles database access errors.

Steps in Database Connectivity

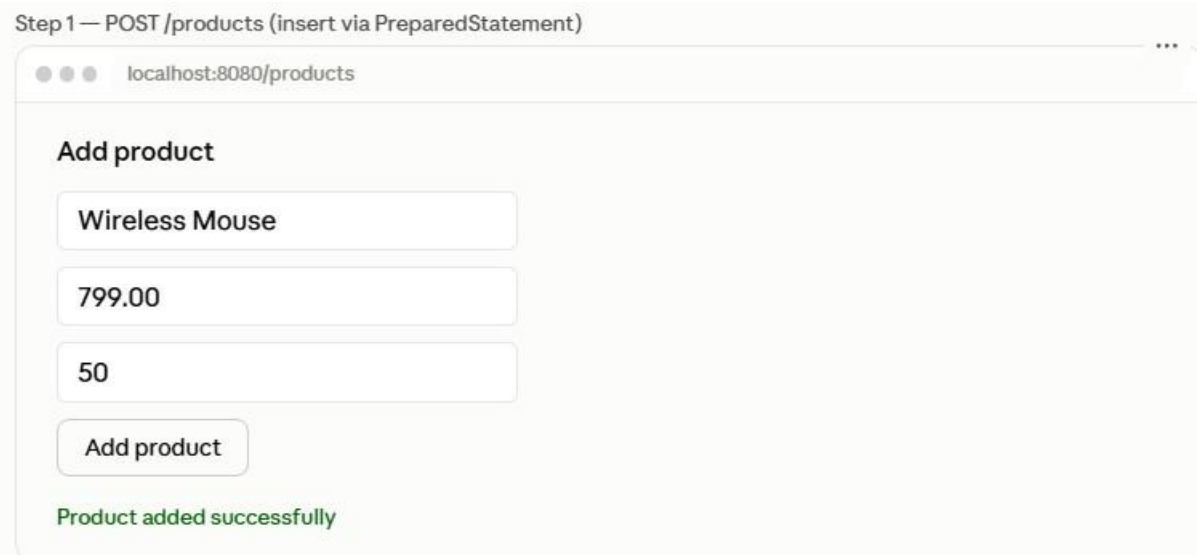
1. **Load/register the driver** (auto-loaded via JDBC 4.0+ SPI, but can be explicit).
2. **Establish a connection** using `DriverManager.getConnection(url, user, password)`.
3. **Create a Statement/PreparedStatement**.
4. **Execute the SQL query** (`executeUpdate()` for INSERT/UPDATE/DELETE, `executeQuery()` for SELECT).
5. **Process the ResultSet** (for SELECT queries).
6. **Close resources** (Connection, Statement, ResultSet) — ideally via try-with-resources.
7. **Handle exceptions** using try-catch around `SQLException`.

Implementation

Database table (MySQL):

```
CREATE TABLE products (
```

```
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL
);
```



DBConnection.java — Connection utility

```
import java.sql.*;

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/ecommerce_db";
```

```
    private static final String USER = "root";
    private static final String PASSWORD = "yourpassword";

    public static Connection getConnection() throws SQLException {
        // Driver auto-registered via JDBC 4.0+ ServiceLoader mechanism
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

ProductDAO.java — Insert and retrieve using PreparedStatement (prevents SQL injection)

```
import java.sql.*;
import java.util.*;

public class ProductDAO {

    // Insert product
    public boolean addProduct(String name, double price, int stock) {
        String sql = "INSERT INTO products (name, price, stock) VALUES (?, ?, ?)";

        try (Connection conn = DBConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, name);
            pstmt.setDouble(2, price);
            pstmt.setInt(3, stock);

            int rowsAffected = pstmt.executeUpdate();
            return rowsAffected > 0;

        } catch (SQLIntegrityConstraintViolationException e) {
            System.err.println("Constraint violation: " + e.getMessage());
            return false;
```

```
    } catch (SQLException e) {
        System.err.println("Database error while inserting product: " + e.getMessage());
        return false;
    }
}

// Retrieve all products
public List<Product> getAllProducts() {
    List<Product> products = new ArrayList<>();
    String sql = "SELECT product_id, name, price, stock FROM products";

    try (Connection conn = DBConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql);
         ResultSet rs = pstmt.executeQuery()) {

        while (rs.next()) {
            Product p = new Product(
                rs.getInt("product_id"),
                rs.getString("name"),
                rs.getDouble("price"),
                rs.getInt("stock")
            );
            products.add(p);
        }

    } catch (SQLException e) {
        System.err.println("Database error while fetching products: " + e.getMessage());
    }
    return products;
}

// Retrieve single product by ID
```

```
    public Product getProductById(int id) {
        String sql = "SELECT product_id, name, price, stock FROM products WHERE
product_id = ?";
        Product product = null;

        try (Connection conn = DBConnection.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, id);

            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    product = new Product(
                        rs.getInt("product_id"),
                        rs.getString("name"),
                        rs.getDouble("price"),
                        rs.getInt("stock")
                    );
                }
            }

        } catch (SQLException e) {
            System.err.println("Error fetching product ID " + id + ": " + e.getMessage());
        }
        return product;
    }
}
```

Product.java — Model class

```
public class Product {
    private int id;
    private String name;
```

```
    private double price;
    private int stock;

    public Product(int id, String name, double price, int stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stock = stock;
    }
    // getters and setters omitted for brevity
    @Override
    public String toString() {
        return "Product{id=" + id + ", name='" + name + "', price=" + price + ", stock=" + stock +
"}";
    }
}
```

ProductServlet.java — Wiring JDBC into the servlet layer

```
@WebServlet("/products")
public class ProductServlet extends HttpServlet {
    private ProductDAO productDAO = new ProductDAO();

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        double price = Double.parseDouble(request.getParameter("price"));
        int stock = Integer.parseInt(request.getParameter("stock"));

        boolean success = productDAO.addProduct(name, price, stock);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```
        out.println(success ? "<p>Product added successfully</p>" : "<p>Failed to add
product</p>");
    }
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        List<Product> products = productDAO.getAllProducts();
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<table
border='1'><tr><th>ID</th><th>Name</th><th>Price</th><th>Stock</th></tr>");
        for (Product p : products) {
            out.println("<tr><td>" + p.getId() + "</td><td>" + p.getName() +
                    "</td><td>" + p.getPrice() + "</td><td>" + p.getStock() + "</td></tr>");
        }
        out.println("</table>");
    }
}
```

Exception Handling Best Practices

1. **Use** `try-with-resources` for `Connection`, `Statement`, and `ResultSet` — ensures they're closed automatically even if an exception occurs, preventing connection leaks.
2. **Catch specific exceptions before generic ones** — e.g., `SQLIntegrityConstraintViolationException` (duplicate keys, null constraint violations) before generic `SQLException`.
3. **Never expose raw SQL error messages to the end user** — log them server-side and show a generic message to the client (avoids leaking schema details — a security concern).
4. **Always use** `PreparedStatement` **instead of** `Statement` with concatenated strings — prevents SQL injection, since parameters are bound safely rather than interpolated into the query text.
5. **Use connection pooling** (e.g., HikariCP, Apache DBCP) in production instead of opening a new `DriverManager` connection per request — raw `DriverManager.getConnection()` per request is expensive and doesn't scale well under load.
6. **Roll back transactions on failure** when performing multiple related writes (e.g., `conn.setAutoCommit(false)`, then `conn.rollback()` in the catch block) to maintain data consistency for multi-step operations like order placement with inventory deduction.